

Lists in Python

This is a collection of tasks to master list understanding and usage. A lot of them are about implementing list methods, so you shouldn't use them until you implement them. I mean, after you write for example a reverse list logic - use method in other cases.

This is considered as a good practice to understand how it exactly works, after that you will be given some arbitrary tasks on lists - feel free to use anything you know about lists.

Contents

1. [Extend](#)
2. [Insert](#)
3. [Remove](#)
4. [Pop](#)
5. [Clear](#)
6. [Index](#)
7. [Reverse](#)
8. [Count](#)
9. [Sort](#)
10. [Poisoning Enemies](#)
11. [Higher dimension](#)
12. [Matrix Addition](#)
13. [Flatten](#)

Extend

Implement the logic of extend method. You are given two lists **a** and **b**. List **a** should be extended with list **b** content. You cant use `a + b` since that same as using extend method.

Input Format

a : list of values.

b : list of values.

Standard input

Output Format

list **a** extended with list **b**

Standard output

Examples

```
1 2 3 4 5
6 7 8 9 10
```

```
1 2 3 4 5 6 7 8 9 10
```

Insert

Implement the logic of insert method. You are given list **a**, value **b** and index **idx** . Value **b** should be inserted in list **a** on index **idx**. That means that index of value **b** inside the list will be equal to **idx**.

Input Format

a : list of values.

b : any value.

idx : integer value.

Standard input

Output Format

list **a** with inserted value **b** on index **idx**.

Standard output

Examples

```
8 1 2 1 43
6
2
```

```
8 1 6 2 1 43
```

```
'Hello' 28 10 22 10 10 10
'What'
7
```

```
'Hello' 28 10 22 10 10 10 'What'
```

Remove

Implement the logic of remove method. You are given list **a** and value **b**. The first occurrence of value **b** must be deleted from list.

Input Format

a : list of values.

b : any value.

Standard input

Output Format

list **a** with deleted value **b**.

Standard output

Examples

```
8 1 2 1 43
6
```

```
8 1 6 2 1 43
```

```
'Hello' 28 10 22 10 10 10
10
```

```
'Hello' 28 22 10 10 10
```

Pop

Implement the logic of pop method. You are given list **a** and index **idx**. Element of list **a** on index **idx** must be removed. Also print the deleted value.

Input Format

a : list of values.

idx : integer value.

Standard input

Output Format

list **a** with deleted element on index **idx**.

deleted value : any value or None if not exists.

Standard output

Examples

```
8 1 2 1 43
6
```

```
8 1 6 2 1 43
None
```

```
'Hello' 28 10 22 10 10 10
0
```

```
28 22 10 10 10 10
'Hello'
```

Clear

Implement the logic of clear method. You are given list **a**. You should remove every element of this list.

Input Format

a : list of values.

Standard input

Output Format

empty list.

Standard output

Examples

```
8 1 2 1 43
```

```
[]
```

```
'Hello' 28 10 22 10 10 10
```

```
[]
```

Index

Implement the logic of index method. You are given list **a**, value **b**, and two indexes **start** and **end**. You should find index of value **b** inside list **a** between **start** and **end**.

Input Format

a : list of values.
b : any value.
start : integer value.
end : integer value.
Standard input

Output Format

Index of value : any value or None if not exists.
Standard output

Examples

```
8 1 2 1 43
2
1
3
```

```
2
```

```
'Hello' 28 10 22 10 10 10
333
4
6
```

```
None
```

Reverse

Implement the logic of reverse method. You are given list **a**, it should be reversed in place - that means that there should not be any copy, the list **a** itself should become reversed.

Input Format

a : list of values.

Standard input

Output Format

list.

Standard output

Examples

```
8 1 2 1 43
```

```
43 1 2 1 8
```

```
'Hello' 28 10 22 10 10 10
```

```
10 10 10 22 10 28 'Hello'
```


Count

Implement the logic of count method. You are given list **a** and value **b**. Count how many time value **b** appears in given list.

Input Format

a : list of values.

b : any value.

Standard input

Output Format

Count of value **b** : integer value.

Standard output

Examples

```
8 1 2 1 43
6
```

```
0
```

```
'Hello' 28 10 22 10 10 10
10
```

```
4
```

Sort

Implement the logic of sort method. You are given list **a**, it should be sorted in some order - you can choose.

There are many many algorithms for doing this. You can start with bubble or insertion sort - you can google them. After this you can try implementing quick and merge sort.

After you have implemented some amount of sort algorithms - you can benchmark them to see the difference in efficiency.

Input Format

a : list of values.
Standard input

Output Format

list.
Standard output

Examples

```
8 1 2 1 43
```

```
1 1 2 8 43
```

```
'Hello' 28 10 22 10 10 10
```

```
We cant sort this list, so it stays the same
```

```
3 8 20 46 1 35 2 2 4 6
```

```
1 2 2 3 4 6 8 20 35 46
```

Poisoning Enemies

There are 10 enemies, each one has 10 health point initially.

Enemy can become poisoned. Being poisoned means an enemy loses 1 hp every 1 second.

Given a time in seconds and some additional commands calculate enemies hp values by the end of time.

There is no work with time actually by the way.

Input Format

s seconds : integer value.

Every second you receive a command - Integer n - number of enemy.

If this enemy is not poisoned it becomes poisoned.

If this enemy is poisoned it becomes normal.

If command -1 is given - nothing happens.

Standard input

Output Format

Health of enemies : list.

Standard output

Examples

```
5
1
2
2
-1
4
```

```
10 5 9 10 9 10 10 10 10 10
```

Higher dimension

Did you know you can store a list inside a list? Well - you can, now try it out and print a result.

Two dimensional list is also called a table or matrix. Create a table of 4 by 3 size (4 - number of rows, 3 - number of columns) and fill it with zeros.

Now iterate through it and change every value to a sum of its indices.

Output Format

A table.

Standard output

Examples

Input: none

```
0 1 2
1 2 3
2 3 4
3 4 5
```

Notice that you should print it out not like `[[1, 2, 3], [2, 3, 4]]` this.

Matrix addition

Now you have to implement a matrix addition logic. Having two matrixes of same sizes you can perform an addition operation by calculating a sum between values of same positions.

Matrixes are big, it is not very handy to type them in terminal for testing your program - so lets read them from file.

Having a file named input.txt you can open it like this:

```
with open("input.txt", "r", encoding="utf-8") as file:
```

```
    fileContent = file.read()
```

There are other file methods to read from file and write to it, you can look them up in python documentation.

Input Format

Two numbers - sizes of matixes. Two matrixes of give sizes.

Input file: input.txt

Output Format

Output file: output.txt

Examples

```
bInput input.txt:
```

```
2 2
```

```
1 1
```

```
1 1
```

```
4 4
```

```
5 5
```

```
5 5
```

```
6 6
```

Try to make matrix1 and matrix2 stay unchanged, explore how to copy a normal list and nested list. Try different ways and print all matrixes to see the results.

Flatten

You are given a list of lists aka table. Flattening is transforming higher dimension list to lower dimension list. You should transform a list of dimension 2 into list of dimension 1.

Input Format

a : list of lists.

Input file: input.txt

Output Format

list.

Standard output

Examples

```
[[8, 1], [2, 1], [43, 55]]
```

```
[8, 1, 2, 1, 43, 55]
```